

COMS 4705.H Natural Language Processing: Midterm

1. Perplexity (10 points) Consider the following (tokenized) poem by Alexander Pushkin (1799-1837). For simplicity, you may assume that the whole poem is one sentence.

If by life you were deceived ,
Don't be dismal , don't be wild !
In the day of grief, be mild
Merry days will come , believe .
Heart is living in tomorrow ;
Present is dejected here ;
In a moment , passes sorrow ;
That which passes will be dear .

Recall that the perplexity of a language model on a corpus is defined as

$$\text{PP} = 2^{-\frac{1}{M} \sum_{i=1}^M \log p(x_i | x_1 \dots x_{i-1})}$$

where M is the number of words plus the STOP symbol in the corpus, and $p(x_i | x_1 \dots x_{i-1})$ is the probability of seeing word x_i given $x_1 \dots x_{i-1}$ defined under the model. We assume that the log base is 2.

Question A (5 points). Suppose the model assigns $p(x_i | x_1 \dots x_{i-1}) = 1$ for every word in the poem (plus the STOP symbol). What is the model's perplexity on the poem?

Solution. 1

Question B (5 points). Suppose the model assigns $p(x_i | x_1 \dots x_{i-1}) = 0.01$ for every word in the poem (plus the STOP symbol). What is the model's perplexity on the poem?

Solution. 100

2. Backward algorithm (10 points) Assume a bigram HMM with parameters

- $t(y'|y)$: probability of tag $y' \in L \cup \{\text{STOP}\}$ given tag $y \in L \cup \{\ast\}$
- $o(x|y)$: probability of word $x \in V$ given tag $y \in L$

We saw that the **forward algorithm** can be used to calculate

$$\alpha(y, i) = \sum_{y_1 \dots y_i \in L^i: y_i = y} p(x_1 \dots x_i, y_1 \dots y_i)$$

for any given sentence $x_1 \dots x_m$. The forward algorithm is given by

1. For each $y \in L$: set $\alpha(y, 1) = t(y|\ast) \times o(x_1|y)$.
2. For $i = 2 \dots m$:
 - (a) For each $y' \in L$:

$$\alpha(y', i) = \sum_{y \in L} \alpha(y, i-1) \times t(y'|y) \times o(x_i|y')$$

Question A (5 points). The **backward algorithm** calculates

$$\beta(y, i) = \sum_{y_i \dots y_m \in L^{m-i+1}: y_i = y} p(x_{i+1} \dots x_m, y_{i+1} \dots y_m | y_i)$$

Complete the pseudocode of the backward algorithm below:

1. For each $y \in L$: set $\beta(y, m) = t(\text{STOP}|y)$.
2. For $i = m-1 \dots 1$:
 - (a) For each $y \in L$:

$$\beta(y, i) =$$

Solution.

$$\beta(y, i) = \sum_{y' \in L} t(y'|y) \times o(x_{i+1}|y') \times \beta(y', i+1)$$

Question B (5 points). Express the **marginal tag bigram probability**

$$B(y, y', i) = \sum_{y_1 \dots y_m \in L^m: y_i = y, y_{i+1} = y'} p(x_1 \dots x_m, y_1 \dots y_m)$$

as a function of α and β .

Solution.

$$B(y, y', i) = \alpha(y, i) \times t(y'|y) \times o(x_{i+1}|y') \times \beta(y', i+1)$$

3. Constrained HMM inference (10 points) An important task in information extraction is **named-entity recognition (NER)**. The goal is to extract entities of interest from text. For instance, given sentence **John Smith works at New York Times**, we wish to identify **John Smith** as an entity of type person and **New York Times** as an entity of type organization:

$$\overbrace{\text{John Smith}}^{\text{PER}} \text{ works at } \overbrace{\text{New York Times}}^{\text{ORG}}$$

One way to tackle this task is to frame it as a tagging problem. Let $E = \{\text{PER}, \text{ORG}, \dots\}$ denote the set of entities. For each entity $e \in E$, we have two tags: $\text{B-}e$ indicating the *beginning* and $\text{I-}e$ indicating the *inside* of this entity. Additionally, every word not part of an entity is tagged with O (others). The above sentence is then tagged as

John/B-PER Smith/I-PER works/O at/O New/B-ORG York/I-ORG Times/I-ORG

This is now just a tagging problem (like POS tagging) with $2|E| + 1$ labels

$$L = \{\text{B-}e : e \in E\} \cup \{\text{I-}e : e \in E\} \cup \{\text{O}\}$$

so we can estimate a bigram HMM from training data of tagged sentences and infer tags of a given sentence (from which we can recover detected entities).

Importantly, note that $\text{I-}e$ tags can only follow certain tags. For example, B-PER I-ORG or O I-PER is not valid (whereas B-PER I-PER and O B-PER are valid). We want to *constrain* the Viterbi algorithm so that **for any $e \in E$, tag $\text{I-}e$ follows only either $\text{B-}e$ or $\text{I-}e$** . The following is a Viterbi algorithm for a given sentence $x_1 \dots x_m$ that imposes this constraint on the label sequence inference.

1. For each $y \in L$: set $\pi(y, 1) = t(y|\star) \times o(x_1|y)$.
2. For $i = 2 \dots m$:
 - (a) For each $y' \in L$:

$$\pi(y', i) = \max_{y \in C(y')} \pi(y, i-1) \times t(y'|y) \times o(x_i|y')$$

Question (10 points). Define $C(y') \subseteq L$ in the above algorithm.

Solution.

$$C(y') = \begin{cases} \{y \in L : y = \text{B-}e \vee y = \text{I-}e\} & \text{if } y' = \text{I-}e \text{ for some } e \in E \\ L & \text{otherwise} \end{cases}$$

4. Count of possible trees (10 points) Assume a probabilistic context-free grammar in Chomsky normal form $G = (N, S, \Sigma, R, q)$ where $N = \{\text{NP}, \text{DT}, \dots\}$ is a set of non-terminal symbols, $S \in N$ is a start symbol, $\Sigma = \{\text{the}, \text{school}, \dots\}$ is a set of terminal symbols, R is a set of rules of form either $a \rightarrow b\ c$ where $a, b, c \in N$ or $a \rightarrow x$ where $a \in N$ and $x \in \Sigma$, and $q(r) > 0$ is the probability of rule $r \in R$. Note that we assume nonzero probability $q(r)$ for each $r \in R$ (as would be the case if we estimate the grammar from a treebank).

Given a sentence $x = x_1 \dots x_m$, let $T_G(x)$ denote the set of all possible parse trees under grammar G . Recall that we can use the *inside algorithm* to compute

$$\sum_{\tau \in T_G(x)} p(\tau) \quad (1)$$

where $p(\tau)$ is the probability of tree τ under G . This is achieved by filling the chart

$$\pi(a, i, j) = \sum_{\tau \in T_G(x_i \dots x_j | a)} p(\tau)$$

where $T_G(x_i \dots x_j | a)$ denotes the set of all possible parse trees under grammar G spanning $x_i \dots x_j$ with $a \in N$ as the root. Eq. (1) is then given by $\pi(S, 1, m)$. Here is the inside algorithm:

1. For $i = 1 \dots m$, for each $a \in N$:

$$\pi(a, i, i) = \begin{cases} q(a \rightarrow x_i) & \text{if } a \rightarrow x_i \in R \\ 0 & \text{otherwise} \end{cases}$$

2. For $l = 2 \dots m$, for $i = 1 \dots m - l$:

(a) Let $j = i + l$.

(b) For each $a \in N$,

$$\pi(a, i, j) = \sum_{\substack{a \rightarrow b\ c \in R \\ i \leq k < j}} q(a \rightarrow b\ c) \times \pi(b, i, k) \times \pi(c, k + 1, j)$$

Question (10 points). Modify the inside algorithm so that we obtain the **number of possible parse trees** of sentence x under grammar G .

Solution.

1. For $i = 1 \dots m$, for each $a \in N$:

$$\pi(a, i, i) = \begin{cases} 1 & \text{if } a \rightarrow x_i \in R \\ 0 & \text{otherwise} \end{cases}$$

2. For $l = 2 \dots m$, for $i = 1 \dots n - l$:

(a) Let $j = i + l$.

(b) For each $a \in N$,

$$\pi(a, i, j) = \sum_{\substack{a \rightarrow b\ c \in R \\ i \leq k < j: \pi(b, i, k) > 0 \wedge \pi(c, k+1, j) > 0}} 1$$

5. Arc-hybrid (15 points) Recall that the arc-standard transition system is defined by the following transitions:

- **SHIFT**: $(\sigma, i|\beta, A) \Rightarrow (\sigma|i, \beta, A)$
- **LEFT_l** for each $l \in L$: $(\sigma|i|j, \beta, A) \Rightarrow (\sigma|j, \beta, A \cup \{(j, l, i)\})$
- **RIGHT_l** for each $l \in L$: $(\sigma|i|j, \beta, A) \Rightarrow (\sigma|i, \beta, A \cup \{(i, l, j)\})$

The arc-eager system is defined by the following transitions:

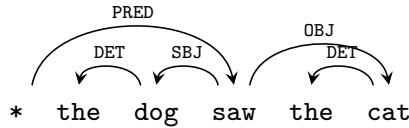
- **SHIFT**: $(\sigma, i|\beta, A) \Rightarrow (\sigma|i, \beta, A)$
- **REDUCE**: $(\sigma|i, \beta, A) \Rightarrow (\sigma, \beta, A)$
- **LEFT_l^e** for each $l \in L$: $(\sigma|i, j|\beta, A) \Rightarrow (\sigma, j|\beta, A \cup \{(j, l, i)\})$
- **RIGHT_l^e** for each $l \in L$: $(\sigma|i, j|\beta, A) \Rightarrow (\sigma|i|j, \beta, A \cup \{(i, l, j)\})$

Now we consider a new system called **arc-hybrid** (Kuhlmann et al., 2011). The arc-hybrid transition system is defined by the following transitions:

- **SHIFT**: $(\sigma, i|\beta, A) \Rightarrow (\sigma|i, \beta, A)$
- **LEFT_l^e** for each $l \in L$: $(\sigma|i, j|\beta, A) \Rightarrow (\sigma, j|\beta, A \cup \{(j, l, i)\})$
- **RIGHT_l** for each $l \in L$: $(\sigma|i|j, \beta, A) \Rightarrow (\sigma|i, \beta, A \cup \{(i, l, j)\})$

Note that it uses the left transition of the arc-eager and the right transition of the arc-standard (hence hybrid).

Question A (5 points). Show a transition sequence from $c_0 = ([0], [1 \dots m], \{ \})$ to $c_T = ([0], [], A)$ under the arc-hybrid system where A corresponds to the following tree:



Solution. **SHIFT**, **LEFT_{DET}^e**, **SHIFT**, **LEFT_{SBJ}^e**, **SHIFT**, **SHIFT**, **LEFT_{DET}^e**, **SHIFT**, **RIGHT_{OBJ}**, **RIGHT_{PRED}**

Question B (5 points). A system is called **top-down** if each node cannot obtain children nodes until it receives a parent, **bottom-up** if each node cannot obtain a parent until it has collected all its children nodes. Is the arc-hybrid system (i) top-down, (ii) bottom-up, or (iii) neither top-down nor bottom-up? Choose exactly one and justify your answer in one sentence.

Solution. Bottom-up: each node is eliminated as soon as it gets a parent.

Question C (5 points). Choose true or false for the statement: in the arc-eager system, a node can collect its children from the left first and then right, and also from the right first and then left. Explain why.

Solution. False: a node must collect its left children first before right since it can only make right children once on the stack.

6. Beam Search (20 points) A quadruple HMM has parameters

- $t(y'''|y, y', y'')$: probability of tag $y''' \in L \cup \{\text{STOP}\}$ given consecutive tags $y, y', y'' \in L \cup \{*\}$
- $o(x|y)$: probability of word $x \in V$ given tag $y \in L$

and defines the joint probability distribution

$$p(x_1 \dots x_m, y_1 \dots y_m) = \prod_{i=1}^{m+1} t(y_i | y_{i-3}, y_{i-2}, y_{i-1}) \times \prod_{i=1}^m o(x_i | y_i)$$

where $y_{-2} = y_{-1} = y_0 = *$ and $y_{m+1} = \text{STOP}$ are special start/stop symbols. Assume we have the model parameters t and o .

Question A (5 points). Derive a Viterbi algorithm that fills the chart

$$\pi(y, y', y'', i) = \max_{y_1 \dots y_i \in L^i: y_{i-2}=y, y_{i-1}=y', y_i=y''} p(x_1 \dots x_i, y_1 \dots y_i)$$

for any given sentence $x_1 \dots x_m$.

Solution.

1. For each $y \in L$: set $\pi(*, *, y, 1) = t(y|*, *, *) \times o(x_1|y)$.
2. For $i = 2 \dots m$:

(a) For each $y', y'', y''' \in L$:

$$\pi(y', y'', y''', i) = \max_{y \in L} \pi(y, y', y'', i-1) \times t(y'''|y, y', y'') \times o(x_i|y''')$$

Question B (5 points). What is the asymptotic runtime of the algorithm in Question A?

Solution. $O(m|L|^4)$

Question C (5 points). Write a beam search algorithm for this model with beam size K and explain its runtime complexity. The algorithm must be in terms of parameters t and o . You may assume an implementation of the leaky priority queue q with capacity K that can push/pop an sequence-score pair $(y_1 \dots y_i, s)$ in $O(\log K)$ time.

Solution. $O(K \log K |L| m)$

1. $q \leftarrow \text{leaky_priority_queue}(K)$
2. $q.\text{push}([* * *, 1])$
3. For $i = 1 \dots m$:
 - (a) $\mathcal{B}_{i-1} \leftarrow q.\text{dump}()$
 - (b) For $(y_{-2} \dots y_{i-1}, s) \in \mathcal{B}_{i-1}$:
 - i. If $i < m$:

$$q.\text{push}([y_{-2} \dots y_{i-1} \ y, s \times t(y|y_{i-3}, y_{i-2}, y_{i-1}) \times o(x_i|y)]) \quad \forall y \in L$$
 - ii. Else:

$$q.\text{push}([y_{-2} \dots y_{i-1} \ y, s \times t(y|y_{i-3}, y_{i-2}, y_{i-1}) \times o(x_i|y) \times t(\text{STOP}|y)]) \quad \forall y \in L$$
4. Return $q.\text{dump}()$.

Question D (5 points). In general, for a sentence of length m , how large should we set the beam width K to be in order to guarantee that the best element in the beam is actually the optimal label sequence?

Solution. $K = |L|^{m-1}$ in general, $K = |L|^3$ for the quadruple HMM